
Cours Python

Un langage informatique (ils en existent énormément) permet d'écrire un programme compréhensible par l'humain.

Un programme est une suite d'instructions qui s'exécutent l'une après l'autre.

(Remarque : pour l'instant, deux instructions ne s'exécuteront jamais en même temps)

La machine, ou le programme sera stocké, ne comprend rien d'autre que des 0 et des 1 (le binaire).



Il existe deux type de traduction en binaire du programme

Langage compilé

La compilation est un mécanisme automatisé (transparent pour l'utilisateur) qui permet de transformer votre programme en code binaire, il nécessite l'usage d'un compilateur propre à votre système d'exploitation. La portabilité est complexe mais la rapidité d'exécution est importante.

Langage interprété

Au contraire de la compilation le langage interprété « exécute » chaque ligne les unes après les autres, pas besoin de compilateur, il est néanmoins moins rapide que le langage compilé et nécessite d'installer le langage sur la machine

Le Python est un langage interprété.

Pour la petite histoire

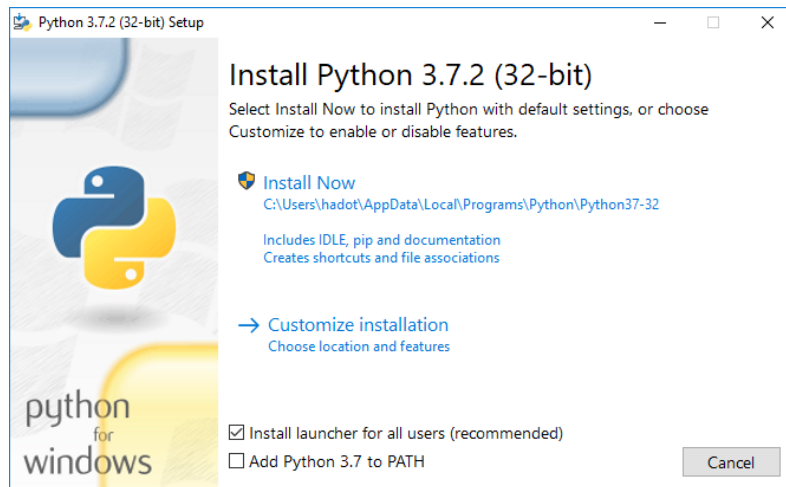
*Python est un langage de programmation, dont la première version est sortie en 1991. Créé par **Guido van Rossum**, il a voyagé du Macintosh de son créateur, qui travaillait à cette époque au Centrum voor Wiskunde en Informatica aux Pays-Bas, jusqu'à se voir associer une organisation à but non lucratif particulièrement dévouée, la **Python Software Foundation**, créée en 2001. Ce langage a été baptisé ainsi en hommage à la troupe de comiques les « Monty Python ». (source Open Classroom)*

Installation de Python

L'adresse pour télécharger Python est la suivante :

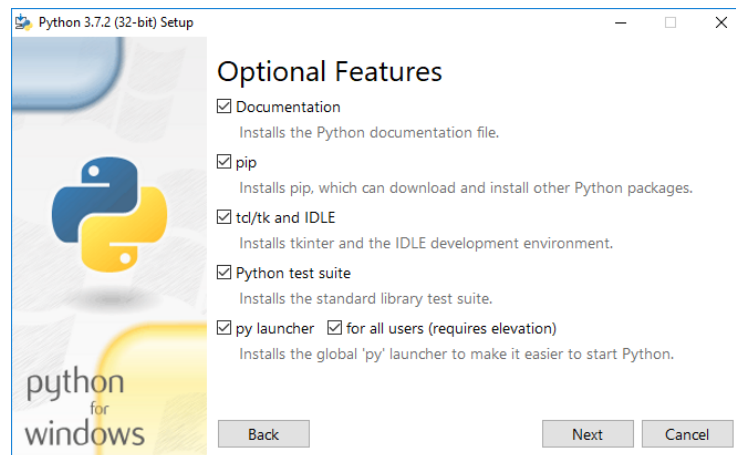
<https://www.python.org/>

Cliquer ensuite sur Downloads et télécharger la dernière version de Python (au 28/01/2019 → python3.7.2) et lancer l'installation.

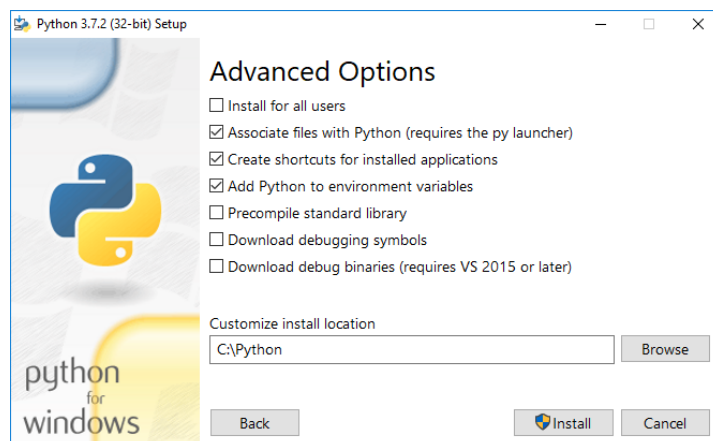


Cliquer sur « Add Python 3.7 to PATH »

Choisir ensuite « **Customize installation** » pour choisir un emplacement plus simple



Cette fenêtre indique les options installées

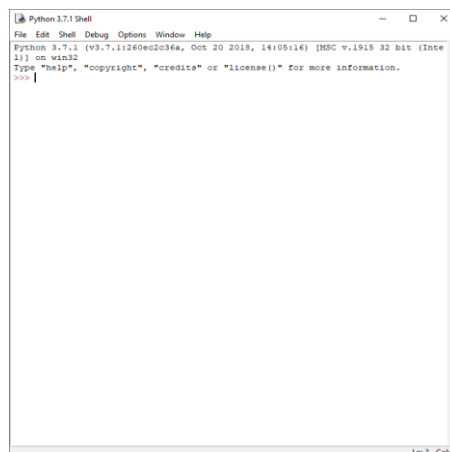


Modifier le répertoire de destination pour un simple **C:\python** puis Install



Lancer ensuite l'interpréteur en cliquant sur IDLE

Vous obtiendrez la fenêtre suivante :



Écrire nos programmes Python dans des fichiers

L'utilisation des lignes de commande devient vite limitée.

Il convient d'utiliser un éditeur de langage pour écrire le programme complet et de le tester ensuite comme une vraie application.

L'éditeur se lance en cliquant sur **File + New File**

Il permet d'écrire des programmes Python

Après avoir écrit le programme enregistrer le au format **.py**

Lancez-le ensuite via IDLE de Python et appuyer sur F5.

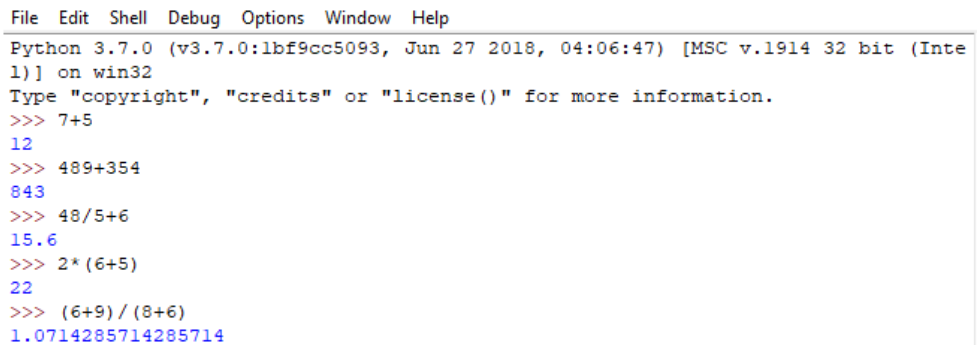
Vous pouvez ajouter une ligne en fin de programme demandant à l'utilisateur s'il veut continuer, ceci permettra à votre fenêtre de ne pas se fermer

`input("Appuyez sur la touche ENTREE pour continuer...")`

Après avoir écrit cette ligne et votre programme vous cliquer deux fois sur le fichier *.py pour l'exécuter

Opérations classiques

Les opérations s'exécutent classiquement, ligne après ligne...

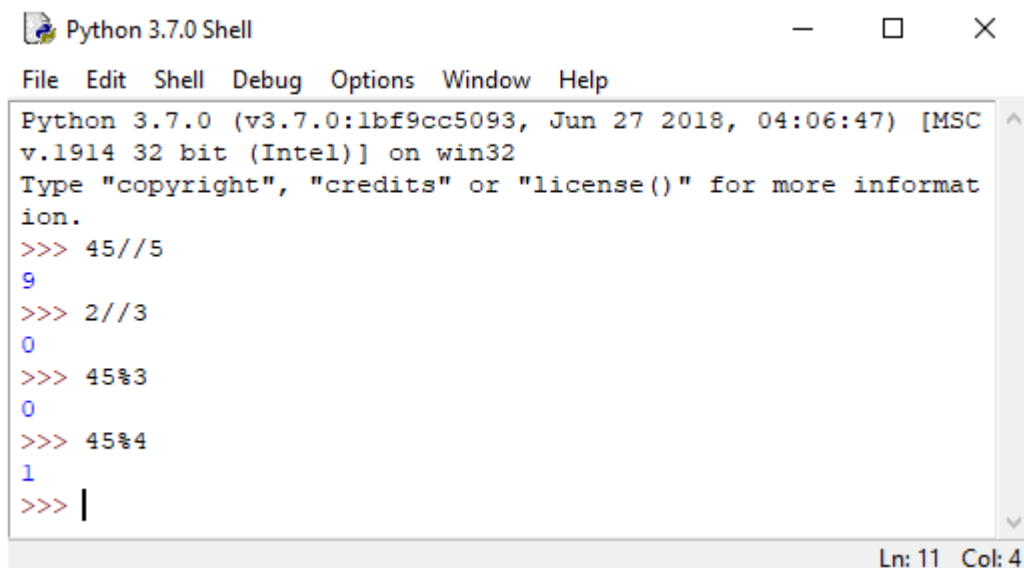


```
File Edit Shell Debug Options Window Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> 7+5
12
>>> 489+354
843
>>> 48/5+6
15.6
>>> 2*(6+5)
22
>>> (6+9)/(8+6)
1.0714285714285714
```

Comme avec une machine à calculer

Opérateurs modulo et partie entière

L'opérateur // récupère la partie entière d'une division et l'opérateur % récupère le reste.



```
Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> 45//5
9
>>> 2//3
0
>>> 45%3
0
>>> 45%4
1
>>> |
```

Ln: 11 Col: 4

Les variables

Une variable est une donnée de votre programme stockée temporairement dans votre mémoire.

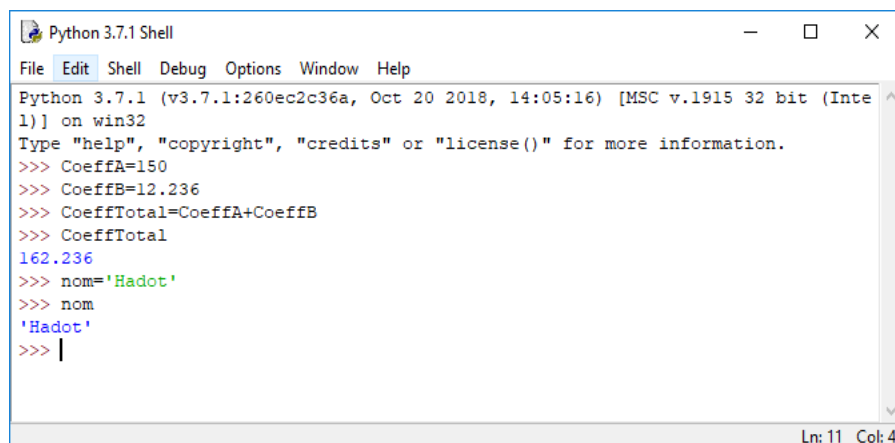
La forme de déclaration est simple :

nomDeLaVariable = valeur

(Convention de nommage : minuscule pour le premier mot majuscule pour la première lettre du mot suivant)

Une variable doit respecter quelques règles de syntaxe incontournables :

1. Le nom de la variable ne peut être composé que de lettres, majuscules ou minuscules, de chiffres et du symbole souligné « _ » (appelé *underscore* en anglais).
2. Le nom de la variable ne peut pas commencer par un chiffre.
3. Le langage Python est sensible à la casse, ce qui signifie que des lettres majuscules et minuscules ne constituent pas la même variable (la variable `AGE` est différente de `aGe`, elle-même différente de `age`).



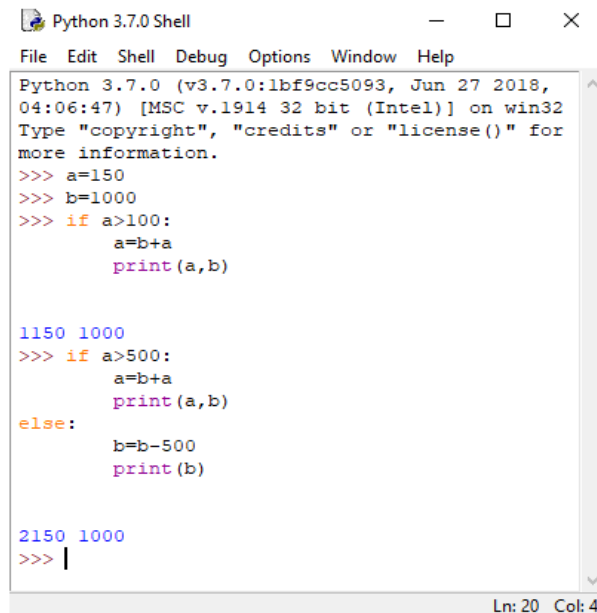
```
Python 3.7.1 Shell
File Edit Shell Debug Options Window Help
Python 3.7.1 (v3.7.1:260ec2c36a, Oct 20 2018, 14:05:16) [MSC v.1915 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> CoeffA=150
>>> CoeffB=12.236
>>> CoeffTotal=CoeffA+CoeffB
>>> CoeffTotal
162.236
>>> nom='Hadot'
>>> nom
'Hadot'
>>> |
```

Structure conditionnelle

- Structure *if, elif et then*

Une structure conditionnelle permet d'effectuer une ou plusieurs lignes si (*if*) une condition est respectée

elif est la contraction de *else if* (sinon si) et permet d'effectuer d'autres lignes de programme si une (et une seule) autre condition est respectée et *then* (alors) permet de clore les tests si aucune condition n'est respectée



```
Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> a=150
>>> b=1000
>>> if a>100:
    a=b+a
    print(a,b)

1150 1000
>>> if a>500:
    a=b+a
    print(a,b)
else:
    b=b-500
    print(b)

2150 1000
>>> |
```

Nous pouvons remarquer que la fonction ***print*** affiche bien les valeurs et que l'indentation (tabulation) est automatique **et** obligatoire (*contrairement à d'autres langages qui pratiquent l'indentation par souci de clarté*)

Opérateurs utilisés

Opérateur	Signification littérale
<	Strictement inférieur à
>	Strictement supérieur à
<=	Inférieur ou égal à
>=	Supérieur ou égal à
==	Égal à
!=	Différent de

Remarque : = pour l'assignation d'une valeur et == pour le test

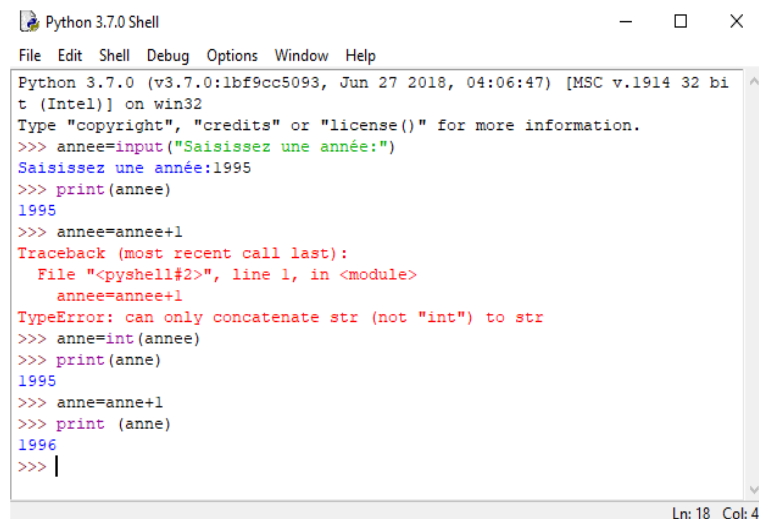
Le rôle d'une « fonction » est fondamental en Python et plus généralement dans tous les langages. Une fonction permet d'écrire une seule fois un bloc d'instructions. Cette fonction sera appelée autant de fois que nécessaire dans le programme. Ainsi on évite d'écrire plusieurs fois les mêmes lignes. Une fonction peut être écrite directement dans le programme principal ou appartenir à une librairie externe.

La fonction input()

La fonction **input()** permet de saisir des caractères au clavier pour ensuite les mémoriser et les utiliser.

La fonction **input()** renvoie le résultat sous la forme d'une chaîne de caractères, pour convertir une chaîne en un nombre il faut utiliser une autre fonction **int()** qui permet de transformer une chaîne en nombre entier (on peut aussi utiliser **float()**, **bool()**).

help("input")



```
Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit
  t (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> annee=input("Saisissez une année:")
Saisissez une année:1995
>>> print(annee)
1995
>>> annee=annee+1
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    annee=annee+1
TypeError: can only concatenate str (not "int") to str
>>> annee=int(annee)
>>> print(annee)
1995
>>> annee=annee+1
>>> print (annee)
1996
>>> |
```

Dans l'exemple ci-dessus on s'aperçoit que la première saisie engendre une erreur à l'incrément (normal la 1^{ère} saisie renvoie une chaîne de caractères et non un nombre), si on convertit cette chaîne avec **int()** l'incrément devient possible.

Les opérateurs logiques

Opérateur	Signification littérale
or	Ou logique
and	Et logique
not	Inversion

Les caractères accentués

Les caractères accentués en Python apparaissent avec une erreur, pour remédier à cela il faut, dès le début de votre programme, ajouter la ligne suivante :

coding: utf-8

Ce qui permet de générer toutes les lettres de tous les langages connus convenablement

Il faut ensuite ajouter un « u » devant les guillemets de la chaîne de caractères :

print(u"Saisir une année:")

Exercice n°1 :

Réaliser un programme permettant de savoir si l'année saisie est bissextile ou non

- Si une année n'est pas multiple de 4, on s'arrête là, elle n'est pas bissextile.
- Si elle est multiple de 4, on regarde si elle est multiple de 100.
 - Si c'est le cas, on regarde si elle est multiple de 400.
 - Si c'est le cas, l'année est bissextile.
 - Sinon, elle n'est pas bissextile.
 - Sinon, elle est bissextile.

Correction en fin de cours

Les structures de boucles

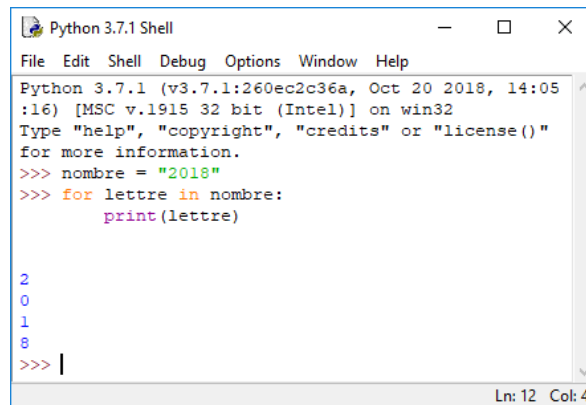
La boucle « while »

Le programme suivant incrémente de 1 une variable i et affiche un texte tant que cette variable n'est pas arrivée à 10 (non inclus).

```
1  # coding: utf-8
2  import os #test
3
4  i=0
5  while i<10:
6      print ("la variable vaut", i)
7      i+=1
8
9  os.system("pause")
10
```


La boucle « for »

La syntaxe est un peu différente de certains langages comme le C++ et le C



```
Python 3.7.1 Shell
File Edit Shell Debug Options Window Help
Python 3.7.1 (v3.7.1:260ec2c36a, Oct 20 2018, 14:05
:16) [MSC v.1915 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license()"
for more information.
>>> nombre = "2018"
>>> for lettre in nombre:
    print(lettre)

2
0
1
8
>>> |
```

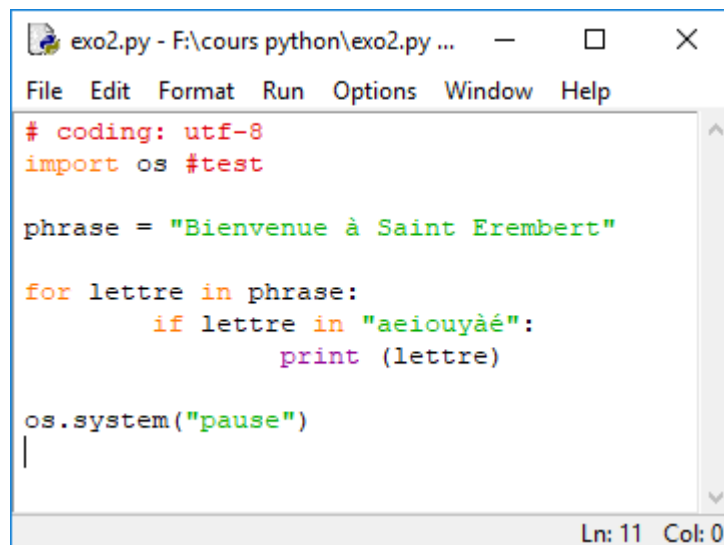
ex1.py

Nombre est une chaîne de caractère (une séquence) la fonction « print » s'exécute à chaque passage d'un caractère

En python on fait appel au mot clé « **in** » qui permet de rechercher dans une séquence les éléments constitutants

Ce mot clé peut être appelé dans des structures différentes

Par exemple si nous voulons détecter les voyelles d'une phrase on écrira :



```
exo2.py - F:\cours python\exo2.py ...
File Edit Format Run Options Window Help
# coding: utf-8
import os #test

phrase = "Bienvenue à Saint Erembert"

for lettre in phrase:
    if lettre in "aeiouyàé":
        print (lettre)

os.system("pause")
|
```

ex2.py

Dans ce cas **phrase** est une séquence de lettre

Pour un nombre on utilisera le mot clé **range**

La boucle **for** peut aussi contenir un argument d'incrémentation

Par exemple :

```
for i in range(0,10,2):
```

0 indique le début du comptage, 10 la fin et 2 l'incrémentation

On peut aussi utiliser un tableau d'items :

```
mylist = [1,2,3,4,5,6,7,8,9,10]
```

Puis :

```
for i in mylist[::2]:  
    print i
```

Les fonctions et modules

Les fonctions permettent de rassembler plusieurs instructions dans un bloc afin de réduire l'écriture d'un programme.

Les modules contiennent plusieurs fonctions.

Pour déclarer une fonction on utilise le mot clé **def**

Déclaration :

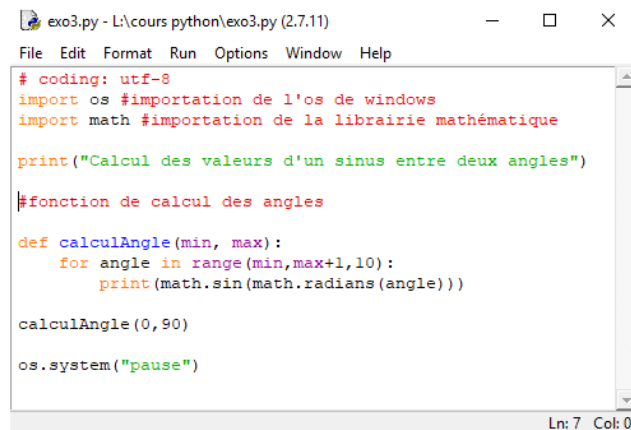
```
def nomDeLaFonction(argument1, argumentN) :
```

argument1, argumentN sont appelé des **paramètres**

Il y a 5 types de fonction :

- *La fonction qui n'a aucun paramètre et qui ne renvoie rien, elle s'exécute c'est tout*
- *La fonction qui accepte un ou plusieurs paramètres mais qui ne renvoie rien*
- *La fonction qui n'accepte pas de paramètre mais qui renvoie une ou plusieurs valeurs (mot clé **return**)*
- *La fonction qui accepte un ou plusieurs paramètres et qui renvoie une ou plusieurs valeurs*
- *La fonction dont on ne connaît pas à l'avance le nombre de paramètres*

L'exemple suivant permet de calculer les valeurs d'un sinus entre deux angles :



```
exo3.py - L:\cours python\exo3.py (2.7.11)
File Edit Format Run Options Window Help
# coding: utf-8
import os #importation de l'os de windows
import math #importation de la librairie mathématique

print("Calcul des valeurs d'un sinus entre deux angles")

#fonction de calcul des angles

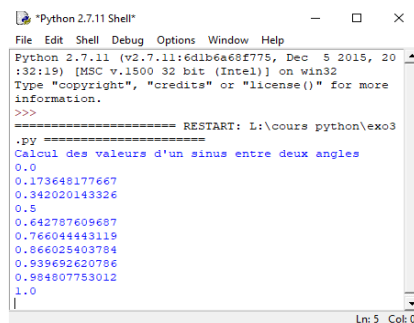
def calculAngle(min, max):
    for angle in range(min,max+1,10):
        print(math.sin(math.radians(angle)))

calculAngle(0,90)

os.system("pause")
Ln: 7 Col: 0
```

ex3.py

Donne :



```
"Python 2.7.11 Shell"
File Edit Shell Debug Options Window Help
Python 2.7.11 (v2.7.11:6d1b6e0f775, Dec 5 2015, 20:32:19) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more
information.
>>>
===== RESTART: L:\cours python\exo3
>>>
.PY
Calcul des valeurs d'un sinus entre deux angles
0.0
0.173648177667
0.342020143326
0.5
0.642787609687
0.766044443119
0.866025403784
0.939692620786
0.984807753012
1.0
|
Ln: 5 Col: 0
```

La librairie **math** est exploitable à l'adresse suivante :

<https://docs.python.org/fr/3.5/library/math.html>

Le mot clé **return** permet de retourner une valeur placée en mémoire vers une variable nommée comme dans l'exemple (ex4.py) suivant :

```
# coding: utf-8

import os #importation de l'os de windows
import math #importation de la librairie mathématique

def calculAngle(angle):
    x=math.sin(math.radians(angle))
    return x

result=calculAngle(-90)
print(result)

os.system("pause")
```

ex4.py

Pour indiquer une valeur par défaut on fixe une des deux valeurs comme suit :

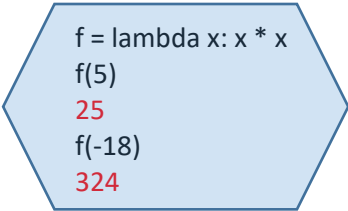
```
def calculAngle(min, max=90)
```

et on appelle la fonction en précisant uniquement le 1^{er} terme

```
calculAngle(0) ;
```

La fonction lambda

La fonction lambda permet de simplifier l'écriture de certaines fonctions par exemple :



```
f = lambda x: x * x  
f(5)  
25  
f(-18)  
324
```

Les modules

Les modules sont des bibliothèques pré-écrites que l'on charge au début d'un programme, on utilise ensuite les fonctions de ces modules.

La syntaxe d'appel est la suivante :

```
import nom_du_module
```

exemple :

```
import math
```

on peut aussi attribuer un autre nom au module en utilisant le mot clé « as » :

```
import math as mathematiques
```

Il est possible de connaître l'intégralité des fonctions d'un module en saisissant la ligne suivante :

```
help (« math »)
```

Autre méthode pour importer des modules :

```
from math import sqrt
```

L'avantage de cette méthode est qu'elle importe que la fonction sqrt et non pas tout le module.

Si vous saisissez :

```
from math import *
```

Vous importez tout le module mais dans ce cas pour appeler une fonction il suffit d'écrire :

```
sqrt(4)
```

Création de module

Nous pouvons reprendre l'exemple du calcul d'un angle pour créer notre fichier module :

```
#fonction de calcul des angles
from math import *

def calculAngle(min, max):
    for angle in range(min,max+1,10):
        print(sin(radians(angle)))
```

Enregistrer ce fichier dans **moduleAngle.py**

Ensuite nous créons notre fichier principal faisant appel au module précédent :

```
# coding: utf-8
import os #importation de l'os de windows
from moduleAngle import *

print("Calcul des valeurs d'un sinus entre deux angles")
calculAngle(0,90)
os.system("pause")
```

On peut aussi appeler le module même si celui-ci est écrit dans le programme principal

Par exemple :

```
# coding: utf-8
import os #importation de l'os de windows

#fonction de calcul des angles
from math import *

def calculAngle(min, max):
    for angle in range(min,max+1,10):
        print(sin(radians(angle)))

# test de la fonction calculAngle

if __name__ == "__main__":
    calculAngle(0,180)

os.system("pause")
```

Package

Un package est un répertoire qui peut contenir d'autres répertoires qui eux même contiennent des modules (.py) qui eux même contiennent des fonctions.

On appelle le package par son nom avec import

Import nomDuPackage

Puis, en séparant par un point, on appelle les différents éléments de ce package

nomDuPackage.nomDuModule.mesureVitesse

Installation d'un module

Pour installer un module Python il est préférable d'être dans le répertoire Python

Cliquer sur Cortana et taper cmd → invite de commande

Ensuite il faut, par ligne de commande DOS, se diriger vers le répertoire Python

La commande « **cd..** » (change directory) permet de remonter dans l'arborescence

La commande « **cd python** » permet de se diriger vers le répertoire « python »

Tapez ensuite la ligne suivante (qui peut être facilement retrouvée sur Internet)

python -m pip install module

(Où module représente le nom du module à installer)

Nous allons, dans l'exemple suivant, créer un fichier Word (docx) avec un texte et une photo.

Nous allons donc saisir la ligne suivante :

python -m pip install python-docx

qui installera la librairie docx pour python

Le programme suivant permet d'insérer une image et un texte dans un fichier docx créé

```
1  import docx
2
3  document = docx.Document()
4
5  p = document.add_paragraph()
6  r = p.add_run()
7  r.add_text('Bonjour à tous je vous présente Saint Erembert')
8  r.add_picture('ste.jpg')
9  r.add_text(' Cela vous plait?')
10
11 document.save('demo.docx')
```

SYNTHESE

Utilisation du module Turtle

<https://zestedesavoir.com/tutoriels/944/a-la-decouverte-de-turtle/>

Comme exemple d'utilisation (synthèse) d'un module nous allons explorer le module Turtle qui permet de dessiner des figures graphiques à la manière d'une tortue

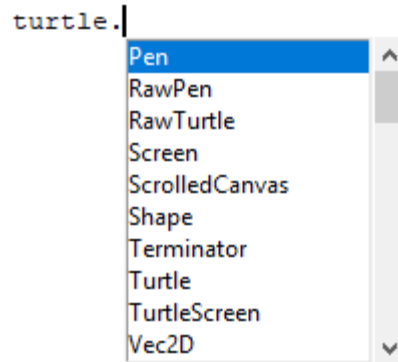
Le module Turtle est installé par défaut dans Python

On importe le module :

`Import turtle`

Le nom turtle est instancié directement

Il suffit alors d'appeler ce nom suivi d'un point pour obtenir toutes les fonctions utiles.



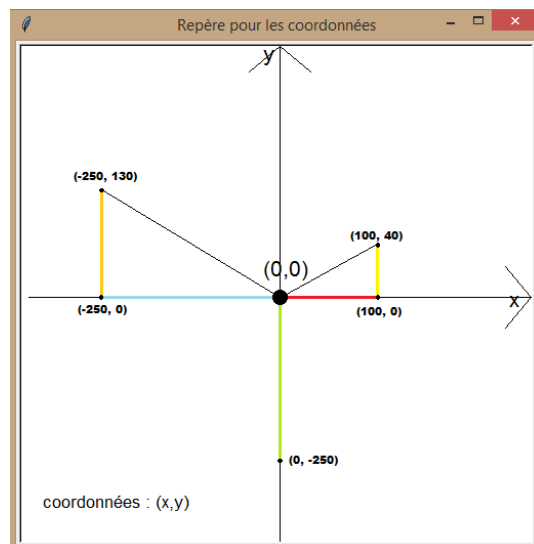
On peut aussi écrire la ligne

`help(« turtle »)`

pour obtenir tous le contenu du module.

Comme vu précédemment une fonction est **toujours** suivie de parenthèses associées à des attributs ou non.

Repère :



Commandes turtle

Ouvrir une fenêtre et donner un nom:

```
turtle.setup(400,400,100,100)
```

```
turtle.title("Graphiques SNT")
```

Rendre invisible le curseur :

```
turtle.hideturtle()
```

Tracer ou non :

```
turtle.up()
```

```
turtle.down()
```

Positionner le curseur :

```
turtle.goto(X,Y)
```

Avancer et reculer :

```
turtle.forward(200)
```

```
turtle.backward(400)
```

Tourner :

```
turtle.left(X)
```

```
turtle.right(90)
```

exercice n°2

Dessiner un quadrillage de 800x600 pixels dans une fenêtre de 1000x800 pixels.

Le quadrillage sera défini par une fonction appelée « quadrillage » ayant comme un attribut un entier appelé « resolution » représentatif du nombre de ligne à tracer (identique en vertical et en horizontal)

exercice n°3

Ajouter la possibilité d'entrée dans une cellule le nombre de lignes avant de dessiner

exercice n°4

Dessiner une courbe de votre choix (par exemple une sinusoïde, une exponentielle, un logarithme ou une hyperbole) en utilisant tout le quadrillage et une couleur différente

Gérer les exceptions

Les exceptions permettent de gérer les erreurs en les anticipant.

Si nous exécutons le code suivant :

```
>>> 3/0
```

Il en résultera l'erreur suivante :

```
Traceback (most recent call last):  
  File "<pyshell#0>", line 1, in <module>  
    3/0  
ZeroDivisionError: division by zero
```

ZeroDivisionError indique l'erreur retourné par Python

L'adresse suivante indique toutes les exceptions de Python

<https://docs.python.org/3/library/exceptions.html>

Nous pouvons aussi imaginer un programme qui demande à l'utilisateur de saisir un nombre, si ce dernier saisit un caractère Python lèvera une erreur, on peut donc essayer de renseigner l'utilisateur sur ce qu'il ne doit pas faire.

La forme minimale est la suivante :

```
try:  
    # Bloc à essayer  
except:  
    # Bloc qui sera exécuté en cas d'erreur
```

L'exemple suivant permet de détecter une erreur de saisie :

```

1  annee=input("saisir un nombre")
2
3  ▼ try:
4      #on essaye de convertir un nombre saisi
5      int(annee)
6      print(annee)
7  ▼ except:
8      #si ce n'est pas un nombre
9      print("vous n'avez pas saisi un nombre")
10
11  input("Appuyez sur la touche ENTREE pour continuer...")

```

Si nous reprenons l'exemple de la fraction x/y par exemple, plusieurs erreurs peuvent se produire

- On peut saisir des lettres au lieu de chiffres
- On peut placer $y=0$

```

1  # coding: utf-8
2
3  x=input("saisir le numérateur")
4  y=input("saisir le dénominateur")
5
6  ▼ try:
7      x=int(x)
8      y=int(y)
9      resultat = x / y
10  except ValueError:
11      print("La variable x ou y possède un type incompatible avec la division.")
12  except ZeroDivisionError:
13      print("La variable y est égale à 0.")
14  ▼ else:
15      print("Le résultat obtenu est", resultat)
16
17  input("Appuyez sur la touche ENTREE pour continuer...")
18  |

```

Voir mots clés **raise**, **finally**, **pass** et **assert**

Objet

Un **objet** est issu d'une **classe**, deux termes essentiels dans la programmation objet.

Un objet est un bloc représentant une entité et possédant des propriétés

L'entité peut être tout ce qu'on imagine :

- une image
- une voiture
- un bâtiment
- un meuble
- un son
- un personnage
- etc..

Chacune de ces entités a forcément des caractéristiques, par exemple une image possède une largeur, une hauteur, une résolution etc... un personnage peut-être de type féminin, masculin, grand, avec des yeux rouges etc...

Dans un programme on dit que l'on **instancie** un objet, c'est-à-dire qu'on le crée et que l'on lui donne un nom.

Dès que cet objet est créé on peut appeler les méthodes qui lui sont liées contenues dans la classe

Exemple de classe

Si nous écrivons tout simplement :

```
>>> x=5
>>> type(x)
<class 'int'>
```

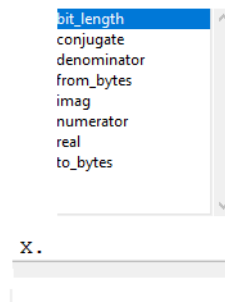
Le mot clé « type » nous renvoie la classe utilisée, en l'occurrence 'int'

De même :

```
>>> Tableau=[1,0,2,3]
>>> type(Tableau)
<class 'list'>
```

A partir du moment où nous avons déclaré une variable, nous avons instancié un objet

Cet objet possède plusieurs méthodes, il suffit de taper un point pour les connaître toutes:



Par exemple la classe 'int' possède une méthode qui permet de retourner le nombre de bit nécessaire pour coder un nombre :

```
>>> X=65535
>>> print(X.bit_length())
>>> 16
```

Il est possible, comme d'habitude, de connaître le contenu de la classe 'int' → `help("int")`

La classe « **str** » est intéressante parce qu'elle permet de manipuler des chaînes de caractères

Voici quelques exemples :

```
>>> Mot="bonjour"
>>> Mot.upper()
'BONJOUR'
>>> Mot.capitalize()
'Bonjour'
>>> Mot=" Bonjour "
>>> Mot.strip()
'Bonjour'
>>> Mot="Bonjour"
>>> Mot.center(25)
'          Bonjour          '
>>>
```

Affichage avec formatage

Le formatage permet d'insérer des éléments dans une chaîne de caractères

On écrit le texte entouré de guillemets comme d'habitude en ajoutant des accolades qui indiquent la position des mots, on ajoute un point suivi du mot clé **format()** avec à l'intérieur les noms des différents mots insérés, l'ordre est déterminant.

Par exemple :

```
>>> Nom=HADOT
>>> Prenom=Manuel
>>> age=57
>>> print("Je suis Mr {0}, mon prénom est {1} et j'ai {2} ans.".format(Nom,Prenom,Age))
```

Donne :

```
===== RESTART: L:/cours python/petits tests/format.py =====
Je suis Mr HADOT, mon prénom est Manuel et j'ai 57 ans.
>>> |
```

On peut aussi utiliser le formatage comme suit :

```
1 # formatage d'une adresse
2
3 adresse = ""
4 {Etablissement}{no_rue}{nom_rue}{code_postal}{nom_ville}({pays})
5
6 """.format(Etablissement="LGT Saint Erembert\n",no_rue=5, nom_rue="rue Salomon Reinach\n", code_postal=78100, nom_ville=" Saint Germain en Laye\n", pays="France")
7
8 print(adresse)
```

Donnera :

```
LGT Saint Erembert
5,rue Salomon Reinach
78100 Saint Germain en Laye
(France)
```

On peut parcourir les caractères d'une chaîne comme avec un tableau

```
>>> chaine="Saint Erembert"
>>> print(chaine[1])
a
>>> print(chaine[0:5])
Saint
>>> print(chaine[5:14])
Erembert
>>> X=chaine[0:5]
>>> Y=chaine[6:14]
>>> print(Y+X)
ErembertSaint
>>> print(Y+" "+X)
Erembert Saint
>>>
```

On peut obtenir la longueur de la chaîne :

```
>>> chaine="Saint Erembert"
>>> print(len(chaine))
14
```

Les listes

La gestion des listes est très importante dans beaucoup d'applications :

- liste de noms (en ajouter, en retirer..)
- listes d'éléments d'une application (des photos en moins en plus....)
- listes d'objets d'un jeu (des clés en plus en moins ...)
- etc...

Une liste est un tableau de variables ou d'objets qui peuvent être mélangés.

C'est une classe de type 'list'

La déclaration est la suivante :

```
MaListe=[0,1,2,"toto",3.2]
```

Si nous écrivons ce petit programme :

```
>>> MaListe=[0,1,2,"toto",3.2]
>>> for i in range(len(MaListe)):
    print(type(MaListe[i]))
```

Nous obtenons :

```
<class 'int'>
<class 'int'>
<class 'int'>
<class 'str'>
<class 'float'>
>>>
```

Comme dans la classe 'str' il est possible de manipuler facilement les éléments d'une liste :

On peut inter changer des éléments :

```
MaListe[2]=MaListe[4]
```

```
>>> MaListe=[0,1,2,"toto",3.2]
>>> X=MaListe[2]
>>> Y=MaListe[4]
>>> MaListe[2]=Y
>>> MaListe[4]=X
>>> print(MaListe)
[0, 1, 3.2, 'toto', 2]
>>>
```

'append' → ajoute un élément en fin de liste

'insert' → insère un élément dans une liste

'+' → concaténation de deux listes

'del' → supprime un élément par un indice

'remove' → supprime un élément par son nom

Exemples :

```
>>> MaListe=[0,1,2,3]
>>> MaListe.append(4)
>>> print(MaListe)
```

```

[0, 1, 2, 3, 4]
>>> MaListe.insert(2,100)
>>> print(MaListe)
[0, 1, 100, 2, 3, 4]
>>> MaListe2=[20,30,40,50]
>>> MaListe=MaListe+MaListe2
>>> print(MaListe)
[0, 1, 100, 2, 3, 4, 20, 30, 40, 50]
>>> del MaListe[4]
>>> print(MaListe)
[0, 1, 100, 2, 4, 20, 30, 40, 50]
>>> MaListe.remove(100)
>>> print(MaListe)
[0, 1, 2, 4, 20, 30, 40, 50]
>>>

```

La fonction enumerate

Ecrivons, par exemple, un petit programme permettant de ranger dans un tableau les lettres de l'alphabet

```

>>> MaListe=[]
>>> for i in range(26):
    MaListe.insert(i,chr(0x41+i))

```

#création d'une liste vide
#boucle de 26 caractères ASCII

Si nous écrivons :

```

>>> for i in enumerate(MaListe):
    print(i)

```

Nous obtenons :

```

(0, 'A')
(1, 'B')
(2, 'C')
(3, 'D')
(4, 'E')
(5, 'F')
(6, 'G')
(7, 'H')
(8, 'I')
(9, 'J')
(10, 'K')
(11, 'L')
(12, 'M')
(13, 'N')
(14, 'O')
(15, 'P')
(16, 'Q')
(17, 'R')
(18, 'S')
(19, 'T')
(20, 'U')
(21, 'V')
(22, 'W')
(23, 'X')
(24, 'Y')
(25, 'Z')
>>>

```


Nous pouvons aussi capturer les deux éléments de la liste :

```
>>> for i, lettre in enumerate(MaListe):  
    print("La lettre {} est la {}e de l'alphabet.".format(lettre, i+1))
```

Ce qui donne :

```
La lettre A est la 1e de l'alphabet.  
La lettre B est la 2e de l'alphabet.  
La lettre C est la 3e de l'alphabet.  
La lettre D est la 4e de l'alphabet.  
La lettre E est la 5e de l'alphabet.  
La lettre F est la 6e de l'alphabet.  
La lettre G est la 7e de l'alphabet.  
La lettre H est la 8e de l'alphabet.  
La lettre I est la 9e de l'alphabet.  
La lettre J est la 10e de l'alphabet.  
La lettre K est la 11e de l'alphabet.  
La lettre L est la 12e de l'alphabet.  
La lettre M est la 13e de l'alphabet.  
La lettre N est la 14e de l'alphabet.  
La lettre O est la 15e de l'alphabet.  
La lettre P est la 16e de l'alphabet.  
La lettre Q est la 17e de l'alphabet.  
La lettre R est la 18e de l'alphabet.  
La lettre S est la 19e de l'alphabet.  
La lettre T est la 20e de l'alphabet.  
La lettre U est la 21e de l'alphabet.  
La lettre V est la 22e de l'alphabet.  
La lettre W est la 23e de l'alphabet.  
La lettre X est la 24e de l'alphabet.  
La lettre Y est la 25e de l'alphabet.  
La lettre Z est la 26e de l'alphabet.  
>>>
```

La fonction split

La fonction split permet de ranger les éléments dans une liste à partir d'une chaîne de caractères et d'éléments séparateurs

```
>>> texte="Bonjour les amis. Je suis à la recherche de mon chat"  
>>> texte=texte.split(" ") # séparation par espace  
>>> for i in enumerate(texte):  
    print(i)
```

On obtient :

```
(0, 'Bonjour')  
(1, 'les')  
(2, 'amis.')  
(3, 'Je')  
(4, 'suis')  
(5, 'à')  
(6, 'la')  
(7, 'recherche')  
(8, 'de')  
(9, 'mon')  
(10, 'chat')
```

On peut aussi rechercher les phrases :

```
>>> texte="Bonjour les amis. Je suis à la recherche de mon chat"  
>>> texte=texte.split(".") # séparation par un point
```

```
>>> for i in enumerate(texte):  
    print(i)
```

On obtient :

```
(0, 'Bonjour les amis')  
(1, ' Je suis à la recherche de mon chat')  
>>>
```

*Remarque : Il existe les **tuples** qui sont des listes immuables, qu'on ne peut modifier. On utilise les parenthèses plutôt que les crochets, on écrira, par exemple :*

*Tuple_vide=(1,2,3) **#cette liste ne pourra être modifiée***

Les fonctions dont on ne connaît pas à l'avance le nombre de paramètres

La déclaration est la suivante :

```
def fonction(*parametres):
```

Si nous écrivons :

```
>>> def fonction_inconnue(*parametres):  
    print("J'ai reçu : {}".format(parametres))
```

```
>>> fonction_inconnue("toto")  
J'ai reçu : ('toto',).  
>>> fonction_inconnue(1,2,3)  
J'ai reçu : (1, 2, 3).  
>>> fonction_inconnue(1,"toto",3.5)  
J'ai reçu : (1, 'toto', 3.5).  
>>> fonction_inconnue(1,"toto",3.5,[4,2,5])  
J'ai reçu : (1, 'toto', 3.5, [4, 2, 5]).  
>>>
```

Par exemple ce programme permet de réaliser la fonction print (en utilisant quand même print impossible de faire autrement) mais cela explique l'usage des fonctions sans paramètre connu et le formatage.

```
>>> def affiche(*parametres):  
    texte="{}".format(parametres)  
    print(texte[2:len(texte)-3])
```

Les compréhensions de liste

Par exemple si on écrit :

```
>>> liste_origine = [0, 1, 2, 3, 4, 5]  
>>> [nb + nb for nb in liste_origine] #on ajoute chaque item de la liste avec lui-même et modifie  
                                     la liste en conséquence
```

On obtient donc :

```
[0, 2, 24, 6]  
>>>
```

On peut faire encore mieux en incluant des conditions :

```
>>> liste_origine = [0, 1, 20, 3, 40, 5]  
>>> [nb for nb in liste_origine if nb<10]  
[0,1,3,5]
```

exercice n°5

On donne la liste suivante :

```
inventaire = [("pommes", 22),("melons", 4),("poires", 18),("fraises", 76),("prunes", 51)]
```

L'objectif de cet exercice est de trier cette liste dans l'ordre décroissant du nombre de fruit pour obtenir ceci :

```
[('fraises', 76), ('prunes', 51), ('pommes', 22), ('poires', 18), ('melons', 4)]
```

- Réaliser cet exercice avec une méthode classique n'utilisant que ce que vous connaissez
- Réaliser cet exercice en utilisant une méthode de type « **sort** » ou « **sorted** » (à vous de chercher comment utiliser cette méthode)

Les dictionnaires

Les dictionnaires sont des objets pouvant en contenir d'autres, à l'instar des listes. Cependant, au lieu d'héberger des informations dans un ordre précis, ils associent chaque objet contenu à une clé (la plupart du temps, une chaîne de caractères).

```
>>> MonDico=dict()
>>> MonDico{"NOM"} = "HADOT"
>>> MonDico{"PRENOM"} = "Manuel"
>>> print(MonDico)
```

Donne : {'NOM': 'HADOT', 'PRENOM': 'Manuel'}

On voit bien que les noms de la liste sont associés à une clé (située en crochets)

Pour accéder à un item :

```
>>> MonDico["NOM"]
'HADOT'
```

On peut créer un dictionnaire :

```
>>> MonDico={"nom":HADOT, "prenom":Manuel, "age":57 }
```

Supprimer un élément :

```
>>> del MonDico["prenom"]
```

Ce qui donnera :

```
{'nom': 'HADOT', 'age': 57}
```

Comparativement au liste les dictionnaires ont une clé qui permet de les traiter, ce qui est plus simple mais oblige à gérer les clés

Parcourir les dictionnaires

Si nous reprenons l'exemple précédent :

```
>>> MonDico={"nom": "HADOT", "prenom": "Manuel", "age":57 }
```

Et que nous écrivons :

```
>>>for i in MonDico :
    print(i)
```

Nous obtenons :

```
nom
prenom
age
```

On s'aperçoit que l'on obtient la liste des clés mais la liste elle-même.
Pour obtenir la liste il faut utiliser le mot clé "**values**" :

```
>>> for i in MonDico.value():  
    print(i)
```

Donnera :

```
HADOT  
Manuel  
57
```

Nous pourrions utiliser une fonction de type **enumerate** pour obtenir les clés et la liste mais en utilisant le mot clé "**items**" :

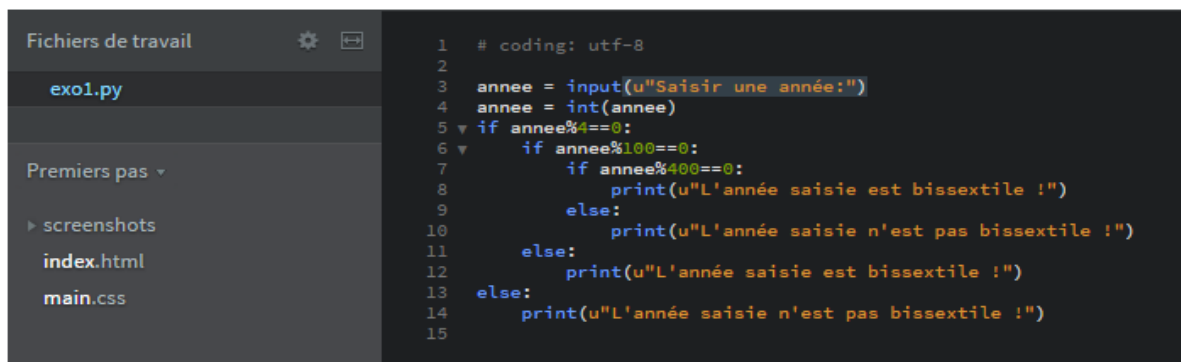
```
>>> for cle, valeur in MonDico.items():  
    print("La clé {} contient la valeur {}".format(cle, valeur))
```

Donnera :

```
La clé nom contient la valeur HADOT.  
La clé prenom contient la valeur Manuel.  
La clé age contient la valeur 57.
```

Les fichiers

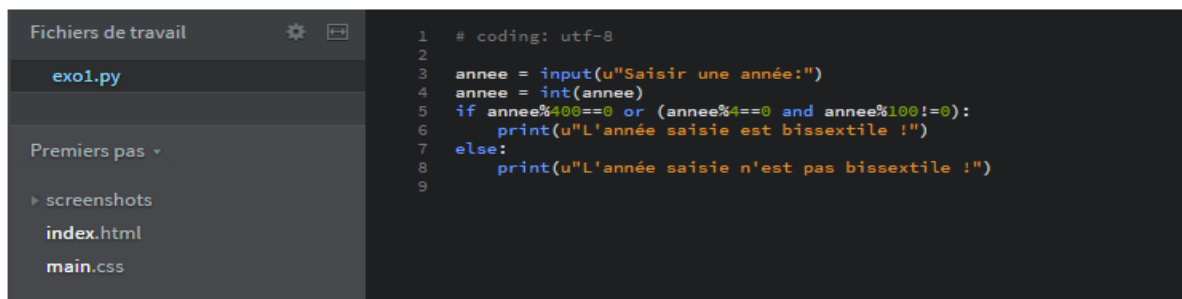
Solution exercice n°1



```
Fichiers de travail  
ex01.py  
  
Premiers pas ▾  
▶ screenshots  
index.html  
main.css
```

```
1  # coding: utf-8  
2  
3  annee = input(u"Saisir une année:")  
4  annee = int(annee)  
5  if annee%4==0:  
6      if annee%100==0:  
7          if annee%400==0:  
8              print(u"L'année saisie est bissextile !")  
9          else:  
10             print(u"L'année saisie n'est pas bissextile !")  
11     else:  
12         print(u"L'année saisie est bissextile !")  
13 else:  
14     print(u"L'année saisie n'est pas bissextile !")  
15
```

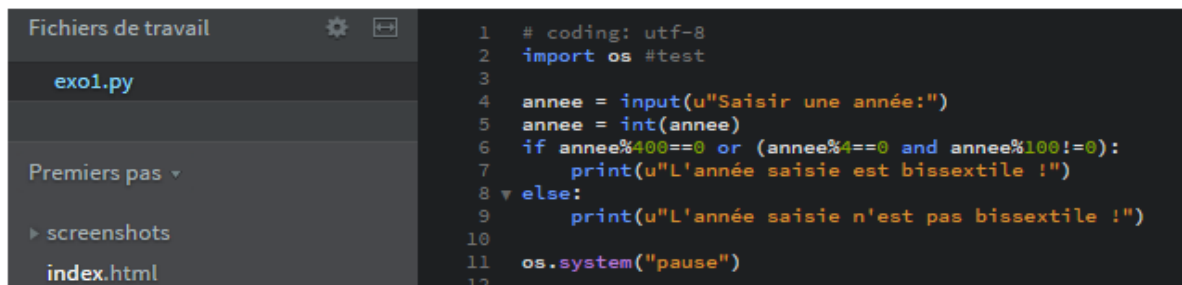
Variante



The screenshot shows a code editor with a sidebar on the left titled "Fichiers de travail" (Working files). It contains a file named "exo1.py" and a folder "Premiers pas" which includes "screenshots", "index.html", and "main.css". The main editor area displays a Python script for checking if a year is a leap year. The script uses the following logic: it prompts the user to enter a year, converts it to an integer, and then checks if it is divisible by 400, or if it is divisible by 4 but not by 100. If it's a leap year, it prints "L'année saisie est bissextile !", otherwise it prints "L'année saisie n'est pas bissextile !".

```
1 # coding: utf-8
2
3 annee = input(u"Saisir une année:")
4 annee = int(annee)
5 if annee%400==0 or (annee%4==0 and annee%100!=0):
6     print(u"L'année saisie est bissextile !")
7 else:
8     print(u"L'année saisie n'est pas bissextile !")
9
```

Et sous Windows



This screenshot is similar to the previous one, but the Python script includes an additional line: `os.system("pause")` at the end, which is used to keep the command window open after the program finishes execution on Windows.

```
1 # coding: utf-8
2 import os #test
3
4 annee = input(u"Saisir une année:")
5 annee = int(annee)
6 if annee%400==0 or (annee%4==0 and annee%100!=0):
7     print(u"L'année saisie est bissextile !")
8 else:
9     print(u"L'année saisie n'est pas bissextile !")
10
11 os.system("pause")
12
```

Solution exercice n°2



The screenshot shows a Python script using the turtle module to draw a grid. It sets up a window titled "Graphiques" with dimensions 1000x800. The script defines a function `quadrillage(resolution)` that draws a grid of lines. It uses `goto` and `forward` to move the turtle to specific coordinates and draw horizontal and vertical lines. The grid is drawn with a resolution of 30.

```
1 import turtle
2 import math
3
4 #Création fenêtre
5 turtle.setup(1000,800,500,150);
6 turtle.title("Graphiques")
7 turtle.hideturtle()
8 turtle.speed(0)
9
10 #fonction quadrillage
11 def quadrillage(resolution):
12     X=600/(resolution-1)
13     for i in range(resolution):
14         turtle.up()
15         turtle.goto(-400, i*X-300)
16         turtle.down()
17         turtle.forward(800)
18     X=800/(resolution-1)
19     turtle.left(90)
20     for i in range(resolution):
21         turtle.up()
22         turtle.speed(0)
23         turtle.goto(i*X-400, -300)
24         turtle.down()
25         turtle.forward(600)
26
27
28 quadrillage(30)
```

Solution exercice n°3

```

1  import turtle
2  import math
3
4  #Création fenêtre
5  turtle.setup(1000,800,500,150);
6  turtle.title("Graphiques")
7  turtle.hideturtle()
8  turtle.speed(0)
9
10 #fonction quadrillage
11 def quadrillage(resolution):
12     X=600/(resolution-1)
13     for i in range(resolution):
14         turtle.up()
15         turtle.goto(-400, i*X-300)
16         turtle.down()
17         turtle.forward(800)
18     X=800/(resolution-1)
19     turtle.left(90)
20     for i in range(resolution):
21         turtle.up()
22         turtle.speed(0)
23         turtle.goto(i*X-400, -300)
24         turtle.down()
25         turtle.forward(600)
26
27
28 nom = int(turtle.textinput("X", "Veuillez saisir un nombre de ligne"))
29 quadrillage(nom)

```

Solution exercice n°4

```

1  import turtle
2  import math
3
4  #Création fenêtre
5  turtle.setup(1000,800,500,150);
6  turtle.title("Graphiques")
7  turtle.hideturtle()
8  turtle.speed(0)
9
10 #fonction quadrillage
11 def quadrillage(resolution):
12     X=600/(resolution-1)
13     for i in range(resolution):
14         turtle.up()
15         turtle.goto(-400, i*X-300)
16         turtle.down()
17         turtle.forward(800)
18     X=800/(resolution-1)
19     turtle.left(90)
20     for i in range(resolution):
21         turtle.up()
22         turtle.speed(0)
23         turtle.goto(i*X-400, -300)
24         turtle.down()
25         turtle.forward(600)
26
27 #fonction signaux
28 def sinusoid():
29     i=-400
30     turtle.pencolor("red")
31     while i<=400:
32         turtle.goto(i, 300*math.sin((2*(i+400)/400)*math.pi))
33         turtle.down()
34         i=i+10
35
36 nom = int(turtle.textinput("X", "Veuillez saisir un nombre de ligne"))
37 quadrillage(nom)
38 turtle.up()
39 sinusoid()

```

exercice n°5.1

```

1 inventaire = [("pommes", 22),("melons", 4),("poires", 18),("fraises", 76),("prunes", 51)]
2
3 X=0
4 n=0
5 final=[]
6
7 while len(inventaire)>0:
8     for i in range(len(inventaire)):
9         if inventaire[i][1]>X:
10             X=inventaire[i][1]
11             n=i
12     final.append(inventaire[n])
13     del inventaire[n]
14     X=0
15 print(final)

```

exercice n°5.2

```

1 inventaire_inverse = [(qtt, nom_fruit) for nom_fruit,qtt in inventaire] #inversion des items pour avoir un nombre
2 inventaire = [(nom_fruit, qtt) for qtt,nom_fruit in sorted(inventaire_inverse,reverse=True)] #tri dans le sens décroissant

```

Deux lignes suffisent !!